

Stable structure learning with HC-Stable and Tabu-Stable algorithms

Neville K. Kitson ^{*}, Anthony C. Constantinou

Machine Intelligence and Decision Systems (MInDS) group, Queen Mary University of London (QMUL), London, E1 4NS, United Kingdom

ARTICLE INFO

Keywords:

Bayesian networks
Algorithm stability
Hill-climbing
Tabu algorithm

ABSTRACT

Many Bayesian Network structure learning algorithms are unstable, with the learned graph sensitive to arbitrary dataset artifacts, such as the ordering of columns (i.e., variable order). PC-Stable [1] attempts to address this issue for the widely-used PC algorithm, prompting researchers to use the ‘stable’ version instead. However, this problem seems to have been overlooked for score-based algorithms. In this study, we show that some widely-used score-based algorithms, as well as hybrid and constraint-based algorithms, including PC-Stable, suffer from the same issue. We propose a novel solution for score-based greedy hill-climbing that eliminates instability by determining a stable node order, leading to consistent results regardless of variable ordering. The new Tabu-Stable algorithms achieve the highest overall performance in terms of mean BIC score, log-likelihood, and structural accuracy across networks. These results highlight the importance of addressing instability in structure learning and provide a robust and practical approach for future applications. This paper extends the scope and impact of our previous work presented at Probabilistic Graphical Models 2024 [2] by incorporating continuous variables, implementing new stable orders that improve performance further, and demonstrating that the approach remains effective in the presence of sampling noise. The implementations, along with usage instructions, are freely available on GitHub at <https://github.com/causal-iq/discovery>.

1. Introduction

Bayesian Networks (BNs) [3] are an approach for modelling complex probabilistic relationships in diverse domains such as health-care [4], fault diagnosis [5] and the environment [6]. They can be used to answer *probabilistic queries* which predict the probability distribution of a subset of variables conditional on the values of other variables, and so can answer questions such as *if these symptoms are present, what is the probability the patient has disease Y?* Moreover, if one additionally assumes that the relationships are *causal*, then the resulting *Causal BN* can be used to answer *interventional queries* such as *if the patient is given this treatment, what is the likely outcome?* [7]. Thus, BN’s have an important potential role as A.I. decision support systems.

Because BNs are probabilistic graphical models, one key challenge is to specify the graphical structure underlying them. Using machine learning to infer this structure from observational data is an active research area. Recent work in [8] shows that many algorithms are *unstable*; that is, sensitive to artifacts of the data, such as the ordering of columns in the data. This paper shows that instability can lead to different learned graphs, resulting in variation in the objective score, model fit, structural accuracy, and learning time. It extends the scope and impact of our previous work presented at Probabilistic Graphical Models 2024 [2] by incorporating

^{*} Corresponding author.

E-mail addresses: n.k.kitson@qmul.ac.uk (N.K. Kitson), a.constantinou@qmul.ac.uk (A.C. Constantinou).

continuous variables and log-likelihood analysis, offering a deeper examination of instability, introducing new algorithmic variants with improved performance, and presenting new results where sampling noise is present.

The principal contribution of this paper is to describe an approach which eliminates this instability for greedy score-based hill-climbing algorithms. Our evaluation focuses on both performance and stability, assessing how well the learned graphs fit the data, how closely they match the true structure, how effectively each algorithm maximises its objective, and how consistently they do so in the presence of dataset artifacts. The Tabu-Stable family of algorithms we introduce is completely insensitive to dataset artifacts and achieves the highest mean BIC score, log-likelihood and structural metrics across categorical and continuous variable networks, including in the presence of sampling noise. The approach described in this paper also has wider applicability to many algorithms that make use of greedy hill-climbing, such as hybrid algorithms.

2. Background

2.1. Bayesian networks

The key element of a Bayesian Network is a Directed Acyclic Graph (DAG) where each node represents a variable and the directed edges, or *arcs*, represent a dependence relationship between the two variables. We denote the n variables in the BN as $X_1, \dots, X_n$. If there is an arc $X_A \rightarrow X_B$, X_A is termed the *parent* of X_B . The DAG is constructed so that a node is conditionally independent of all variables except its descendants given its parents. This is called the *Local Markov Property* and allows the global probability distribution to be expressed compactly as:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \mathbf{Pa}(X_i)) \quad (1)$$

where $\mathbf{Pa}(X_i)$ are the parents of X_i .

A key result flowing from the Local Markov Property is *d-separation*, a graphical condition that identifies variables that are conditionally independent [9]. D-separation can be used to infer the graph structure from the independence and dependence relationships present in the data. In general, however, more than one DAG is consistent with the set of independence relationships present in the data [10]. These *Markov Equivalent* graphs belong to a *Markov Equivalent Class (MEC)*. A MEC is usually represented by a *Completed Partially Directed Acyclic Graph (CPDAG)*, where directed edges indicate edges where all DAGs in the MEC have that same orientation, and undirected edges indicate that some DAGs have one orientation and the rest the other.

The other component of a BN is the specification of the probabilistic dependence relationship between adjacent variables and the probability distributions assumed. For the categorical variable networks considered here, this takes the form of *Conditional Probability Tables (CPTs)* which define a multinomial distribution for the child values for each combination of parental values. For the continuous variable networks considered, it is assumed that the value of a child variable is a linear function of the values of its parents plus a Gaussian noise term. These are termed *Gaussian Bayesian Networks* and the resulting global probability distribution is a multivariate Gaussian.

2.2. Structure learning algorithms

The specification of a BN's DAG structure may be undertaken using human expertise, using a structure learning algorithm to learn it from data, or a combination of both. Structure learning algorithms usually learn from observational data, since it is more readily available. Algorithms typically make assumptions that often do not hold in practice. For example, that there are no missing data, measurement error, or latent confounders.

Constraint-based algorithms such as PC [11], GS [12] and Inter-IAMB [13] use statistical conditional independence (CI) tests to identify the independence relationships in the data, and use the d-separation principle to infer the DAG structure. Constraint-based algorithms usually assume *faithfulness*, which states that there are no independence relationships in the data which are not implied by the DAG. Variants of the approach such as FCI [14] can account for latent confounders. Most constraint-based algorithms aim to return a CPDAG which represents all the possible DAGs that are consistent with the independence and dependence relationships in the data. However, errors arising from the unreliability of the independence tests used or algorithm design may mean that they sometimes return a graph containing directed and undirected edges which is not consistent with any possible DAG. In that case, the learned graph is termed a *non-extendable Partial DAG (PDAG)* [15,14].

Score-based algorithms represent the second major class of algorithms. These follow a more traditional machine-learning approach of using an objective function to assign a score to each graph and then employ some strategy to find a high-scoring graph. The Hill-Climbing (HC) [16] and Tabu [17] algorithms are two simple score-based algorithms that remain competitive and commonly-used. Other score-based approaches search through MEC space, for example, GES [18] and FGES [19]. *Exact* score-based algorithms, such as GOBNILP [20] guarantee to return the highest scoring graph, though typically with limits placed on the number of parents of any node.

Other classes of algorithms include *hybrid* ones such as MMHC [21] and H2PC [22] which use a mix of score and constraint-based methods. More recent developments include algorithms which make additional assumptions about the functional relationships between variables to identify arc orientations [23], and algorithms such as NOTEARS [24] which treat structure learning as a continuous optimisation problem. [25] provides a comprehensive survey across the different classes of algorithms.

The objective function in score-based approaches usually includes the log-likelihood of the data being generated from the graph, with two commonly-used scores being BIC [26] and BDeu [27]. The BIC score, S_{BIC} , for a graph G with n variables and dataset D is computed as follows:

$$S_{BIC}(G, D) = \sum_{i=1}^n \sum_{j=1}^{q_i} \sum_{k=1}^{r_i} \left[N_{ijk} \log \frac{N_{ijk}}{N_{ij}} \right] - \frac{\log N}{2} \cdot F \quad (2)$$

The first term on the right-hand side of Equation (2) is the log-likelihood and is based on counts of values in the dataset. Specifically, N_{ijk} is the number of rows where node X_i has the k^{th} out of the r_i possible values and its parents $\text{Pa}(X_i)$ have the j^{th} combination of values of the q_i possible combinations, and N_{ij} is the total number of rows where the parents have that j^{th} combination of values. The second term is a model complexity penalty, where N is the total number of rows in D , and F is the number of free parameters in the CPTs of the model.

BDeu, S_{BDeu} , is a Bayesian score representing the posterior probability of graph G given the data D assuming some prior beliefs about the probability of each graph and set of parameter values. If all graphs are assumed equally probable, then BDeu is given by:

$$S_{BDeu}(G, D) = \sum_{i=1}^n \sum_{j=1}^{q_i} \left[\log \frac{\Gamma(\frac{N'}{q_i})}{\Gamma(N_{ij} + \frac{N'}{q_i})} + \sum_{k=1}^{r_i} \log \frac{\Gamma(N_{ijk} + \frac{N'}{r_i q_i})}{\Gamma(\frac{N'}{r_i q_i})} \right] \quad (3)$$

where Γ is the Gamma function, N' assigns a weight to the prior parameter beliefs, and other symbols take the same meanings as in Equation (2). BIC and BDeu are *decomposable* scores, since they represent the sum of individual scores for each node. This property facilitates the efficient re-computation of the graph score as the graph is modified. Both are also *score equivalent* which means they assign the same score to all DAGs in a particular MEC.

2.3. Related work

The instability of structure learning algorithms, and specifically their sensitivity to arbitrary dataset artifacts such as *variable order*, seems to have attracted relatively little attention. Variable order in this paper means the order of the columns in the dataset, something which is arbitrary and ideally ought to have no effect on the learned graph. [16] state that “Directions of arcs are dictated by a total order on variables in the database, although the alternative of having the algorithm choose arc directions based on entropy minimization is also available to the user” in their early work on hill-climbing, but there seems to be little subsequent investigation of the impact of these approaches. Exact algorithms, such as that proposed by [28] based on the general-purpose A-Star approach, which implicitly determines the best node order, and those that search over MECs such as GES and FGES should, *in principle*, be insensitive to dataset artifacts. Algorithms that rely on a topological order being specified such as K2 [29], or ones that generate an order themselves [30,31], should also be stable. Approaches that average over several learned graphs, either those sampled from a posterior distribution of graphs such as Order-MCMC [32], or which use different sub-samples of data [33] or different classes of learner [34] might be less sensitive since any effect of dataset artifacts may tend to ‘cancel out’ over the population of learned graphs.

Nonetheless, the stability of algorithms is rarely explicitly considered or evaluated. An exception is PC-Stable [1] where the authors strive to minimise the effect of node processing order in the PC algorithm which they find has a considerable effect on how errors propagate throughout the learning process. PC-Stable offers better accuracy and lower sensitivity to variable order than PC, and it is generally chosen over PC for that reason. However the results in Subsection 5.4 demonstrates that it retains a considerable amount of instability. [8] shows that variable order can impact the ranking of algorithms, but it is not usually considered when comparing algorithms. [35] tries a small number of different orderings in order to improve arc orientations as part of a comparative benchmark, but sensitivity to ordering is not reported.

3. Eliminating instability in hill-climbing

This section discusses the source of instability within hill-climbing algorithms and how it can be addressed. The focus is on the Tabu algorithm since it is commonly used and is competitive in benchmarks, but we apply the same approach to the simpler HC algorithm. The resulting two new algorithms, Tabu-Stable and HC-Stable, are described.

3.1. How instability arises in hill-climbing algorithms

HC (hill-climbing) is perhaps the simplest greedy, score-based algorithm. It typically starts exploring the search-space of graphs from an empty DAG, and searches for the single arc addition, deletion or reversal which increases the objective score of the DAG the most at each iteration. Changes that would create a cycle are not considered. The algorithm terminates when there are no further changes that would increase the score, and the resulting DAG generally has only a locally-maximum score.

Tabu is a higher-performing variant of HC that allows iterations where the score stays the same or decreases allowing the algorithm to escape some local maxima. Tabu maintains a fixed-length list of recently visited DAGs, *tabulist*, to prevent the algorithm from repeatedly considering previously-visited DAGs. The black-coloured pseudo-code in Algorithm 1 shows the main elements of Tabu. The *deltas* variable holds the score change associated with every possible change to the DAG. Lines 5 to 25 form the main iterative loop, with the highest-scoring change for each iteration identified in the **foreach** loop, and applied to the DAG at line 19. *UpdateDeltas*

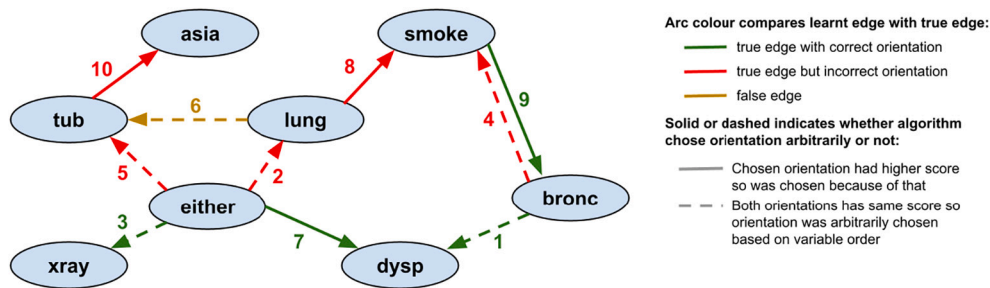


Fig. 1. The sequence of DAG changes when HC learns the categorical Asia network from 10,000 samples. The numbers beside each arc show the iteration at which it is added. Arc colours compare the learned arc against the true arc, and whether it is solid or dashed indicates whether its orientation was arbitrary. Variable order within the dataset is alphabetic. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)

updates *deltas* appropriately following the change; for example, adding arc $A \rightarrow B$ would mean that *deltas* for adding arcs which point to B must be recalculated to take into account that B now has a new parent. The *stop_condition* for the main loop is that none of the last *noinc* (a hyperparameter) changes have increased the score. HC is similar to Algorithm 1 except that *tabulist* is not required, and the *stop_condition* is that there are no further changes which will **increase** the score.

Fig. 1 illustrates the source of instability in hill-climbing by showing the sequence in which HC conventionally learns the Asia network from 10^4 rows of data, when using the BIC score as the objective function. Both orientations of the edge *bronc*—*dysp* give the same maximal score improvement at the first iteration. If HC is implemented to use the variable order to orientate the arc in this situation, and that order is alphabetic, then orientation *bronc* $\rightarrow$ *dysp* would be chosen. This **arbitrary** orientation happens to agree with the true graph. The orientation of the second arc added, *either* $\rightarrow$ *lung*, is similarly arbitrary, but in this case, incorrect. Just like in the case of constraint-based learning with the PC algorithm, **this instability propagates to subsequent iterations**. For example, if the variable order had been such that the second iteration correctly added *lung* $\rightarrow$ *either*, then edge *tub*—*either* would also have been orientated correctly. This, in turn, would have stopped the extraneous arc *lung* $\rightarrow$ *tub* being added. While the impact of variable order varies from network to network, [8] shows that it is generally considerable, typically overshadowing the impact of changing objective functions, sample size or hyperparameters.

3.2. Stabilising hill-climbing

Algorithm 1 illustrates the key elements, shown in red pseudocode, of the new algorithm Tabu-Stable that avoids arbitrary orientations and hence becomes completely insensitive to the ordering of the variables as read from data. The key is to determine a *stable order* at line 4 that is not dependent on the artifacts of the dataset, such as the variable order. The DAG change with the highest score improvement at each iteration is determined in the **for each** loop as usual, but this now also records whether there is an equivalent change (*equiv_change*), which adds an arc in the opposite orientation with the same maximum score improvement. In that case, the one consistent with *stable_order* is added to the DAG. Analogously to the relationship between Tabu and HC, HC-Stable is simply Tabu-Stable with *tabulist* removed and a *stop_condition* that there are no further changes that would **increase** the score.

The function *GetStableOrder* shown in Algorithm 2 returns the stable node order used to avoid arbitrary orientations. This order is generated in two stages. Firstly, lines 1-11 of Algorithm 2 produce *dec_score_order*, which contains nodes primarily ordered by the decomposable objective score, BIC or BDeu for example, that will later be used in the structure learning itself. The sort key for the list has three elements, which in order of precedence are: (1) the score of the node without parents, (2) the mean score of the node where every other node is taken as its single parent (computed in lines 4 to 7), and (3), in the case of categorical variables only, a textual rendition of the counts of values of the variable, e.g. “{‘no’: 5, ‘yes’: 3}”. The first two elements of the sort key use scores such as those described in Equation (2) and Equation (3) and are therefore determined solely by the **data distribution itself**.

Moreover, since the second element of the sort key for a node compares the distribution of combinations of values of that node and every other node, the intuition is that it is very unlikely that two variables will have the same sort key unless they are indeed identical (duplicate). One situation where the first two elements of the sort key are the same for non-identical categorical variables is where the sequences of values for them are isomorphic, e.g. a, a, a, c, c, a and c, c, c, b, b, c , but here the third element of the sort key will differ. If two variables do have the same sort key, then they revert to being ordered by variable order. This will most likely occur because the variables have identical sequences of values; that is, they are effectively duplicate variables. We find that this can occur learning from categorical variables with limited sample sizes. Thus, the whole algorithm retains some unavoidable sensitivity to variable order when there are identical (or duplicate) variables. This is explored in Subsection 5.3.

It is not expected that *dec_score_order* will necessarily be a **good** order, rather, only that it will be insensitive to artifacts of the dataset such as row or column order. [8] shows that if the node ordering is very different to the topological ordering of the true graph, it will likely adversely affect the accuracy of the learned graph. To counter this, lines 12-19 of Algorithm 2 attempt to improve the objective score, which typically also improves structural accuracy, by considering both *dec_score_order* and its reverse. It chooses between these two orders by using HC to learn a graph with each order in lines 13 and 14, to test which results in the higher-scoring DAG. The topological order of the higher-scoring learned graph is returned as *stable_order*. The results in Subsections 5.1 and 5.2 show that this empirical approach stabilises hill-climbing search **and** improves the objective score of the learned graph. It also increases the structural accuracy when learning from categorical variables.

Algorithm 1: Tabu-Stable (changes to standard Tabu shown in red).

Input: *data*, dataset to learn graph from
Output: *best_dag*, highest-scoring DAG found

```

1  stable_order ← GetStableOrder(data) (see Algorithm 2)
2  best_dag ← dag ← empty DAG
3  tabulist ← empty list
4  deltas ← score change for each arc addition
5  repeat
6      max_delta ← None
7      foreach dag_change = AllowedChange(dag, tabulist) do
8          if max_delta = None or delta[dag_change] > max_delta then
9              max_delta ← delta[dag_change]
10             best_change ← dag_change
11             equiv_change ← None
12             else if AddingSameEdgeWithSameDelta(dag_change, best_change) then
13                 equiv_change ← dag_change
14             end if
15         end
16         if equiv_change ≠ None and equiv_change consistent with stable_order then
17             best_change ← equiv_change
18         end if
19         dag ← dag + best_change
20         UpdateDeltas(deltas, best_change)
21         insert dag into tabulist
22         if Score(dag, data) > Score(best_dag, data) then
23             best_dag ← dag
24         end if
25 until stop_condition

```

Algorithm 2: GetStableOrder - determines a stable processing order.

Function *GetStableOrder(data)* :
Input: *data*, dataset learning graph from
Output: *stable_order*, stable order for use in Tabu-Stable

```

1  dec_score_order ← empty_list
2  foreach variable in data do
3      uncond_score = NodeScore(variable)
4      cond_score = 0.0
5      foreach possible single_parent of variable in data do
6          cond_score ← cond_score + NodeScore(variable, single_parent) / (n - 1)
7      end
8      sorted_value_counts ← counts of unique values of variable in data
9      sort_key = (uncond_score, cond_score, sorted_value_counts)
10     dec_score_order ← InsertByKey(variable, sort_key)
11 end
12 inc_score_order ← reverse(dec_score_order)
13 inc_dag ← HC(data, inc_score_order)
14 dec_dag ← HC(data, dec_score_order)
15 if DAGScore(inc_dag, data) > DAGScore(dec_dag, data) then
16     stable_order = TopologicalOrder(inc_dag)
17 else
18     stable_order = TopologicalOrder(dec_dag)
19 end
20 return stable_order

```

4. Evaluation

The stability and accuracy of our approach is assessed using synthetic datasets generated from the seventeen categorical variable networks shown in Table 1 and the seven continuous variable networks shown in Table 2. These networks have between 8 and 109 nodes. The Building network is obtained from the BnRep repository [36], Sports, Covid, Diarrhoea, Property and Formed are obtained from the Bayesys repository [37] and the remainder from the bnlearn repository [38]. These networks are commonly used in the literature to assess algorithms, and are largely expert-specified, generally representing causal networks found in the real world. Sample sizes of 10^2 , 10^3 , 10^4 and 10^5 are used to cover a typical range of sample sizes encountered in practical structure learning, including low-dimensional settings. The BIC score defined in equation (2) is used throughout as the objective function for score-based algorithms and score-based phases of hybrid algorithms.

Table 1
Categorical variable networks used for evaluation.

Network	Subject area	No. of nodes	No. of arcs	Max. in-degree	Max. degree	Free parameters
Asia	Simple patient diagnosis	8	8	2	4	18
Sports	Football match outcomes	9	15	2	7	1,049
Sachs	Protein signalling network	11	17	3	7	178
Covid	COVID in the UK	17	37	5	10	7,834
Child	Lack of oxygen in babies	20	25	2	8	230
Insurance	Car insurance risk	27	52	3	9	1,008
Property	UK property investment	27	31	3	6	3,056
Diarrhoea	Childhood diarrhoea	28	68	8	17	1,716
Water	Waste water treatment	32	66	5	8	10,083
Mildew	Mildew disease in wheat	35	46	3	5	540,150
Alarm	Patient monitoring system	37	46	4	6	509
Barley	Weed control in barley	48	84	4	8	114,005
Hailfinder	Forecasting severe weather	56	66	4	17	2,656
Hepar2	Diagnosing liver disorders	70	123	6	19	1,453
Win95pts	Computer diagnostics	76	112	7	10	574
Formed	Prisoner reoffending risk	88	138	6	11	912
Pathfinder	Lymph-node diagnosis	109	195	5	106	71,890

Table 2
Continuous variable networks used for evaluation.

Network	Subject area	No. of nodes	No. of arcs	Max. in-degree	Max. degree	Free parameters
Sachs	Protein signalling network	11	17	3	7	39
Covid	COVID in the UK	17	37	5	10	71
Building	Damage to building concrete	24	32	4	5	80
Magic-NIAB	Genetic modelling in wheat	44	66	9	10	154
Ecoli70	Gene activity in E. Coli	46	70	4	11	162
Magic-IRRI	Genetic modelling in wheat	64	102	11	11	230
Arth150	Gene activity in a plant	107	150	6	20	364

The learned graphs are evaluated using their log-likelihood, BIC score and their structural metrics. Log-likelihood is a measure of how well the data fits the learned network, and scales with sample size, so we report the normalised log-likelihood; that is, the log-likelihood divided by sample size, which allows comparisons between different sample sizes. We compute log-likelihood on unseen test data, which has the same sample size as the training data but consists of a different random sample drawn from the ground-truth distribution. This provides an indication of how well the learned graph generalises to new data.

The BIC score is a model selection score, defined in Equation (2). Because the Tabu and HC algorithms use BIC as their optimisation criterion in this study, it serves as an appropriate metric for evaluating their success in achieving that objective. As the sample size increases, the log-likelihood component of BIC tends to dominate the complexity penalty, so that BIC *approximately* scales with sample size in our experiments. Hence, we also report the normalised BIC score to facilitate comparisons between experiments with different sample sizes. The BIC score is computed on the same dataset used for learning the graph, since we use it to assess how effectively each algorithm maximises its optimisation objective.

While the mean normalised BIC and log-likelihood for each network tend to stabilise around a characteristic value for that network (see, for example, Table A.17), these values can vary substantially between networks. Therefore, the mean of these scores *across* networks is not meaningful in absolute terms. Nonetheless, it remains useful for observing general trends, such as whether a particular algorithm tends to improve or worsen these scores relative to others across a range of networks. We therefore include the mean normalised BIC and log-likelihood across networks in our analyses to highlight such overall trends.

The structural metrics compare the CPDAG of the true graph with that of the learned graph since only observational data is being used. F1 and the Balanced Scoring Function (BSF) metric proposed in [39] are used as structural metrics. Both of these metrics have the advantage of not scaling with the network size, unlike SHD [21], which facilitates comparing performance across different networks.

F1 ranges between 0.0 to 1.0 and is defined as

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

where Precision and Recall are defined as

$$\text{Precision} = \frac{TP}{TP + FP}, \text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

and TP is the number of True Positives, FP is False Positives and FN is False Negatives. We follow the approach adopted in the bnlearn package [40] to compute these counts as shown in Table 3.

Table 3

The contribution to the True Positive, False Positive, False Negative counts (and hence F1 and BSF) resulting from different combinations of edges in the learned and data-generating graph.

Learned graph	Data-generating graph	True Positive (TP)	False positive (FP)	False Negative (FN)
→	→	1	0	0
—	—	1	0	0
→	no edge	0	1	0
—	no edge	0	1	0
no edge	→	0	0	1
no edge	—	0	0	1
→	←	0	1	1
→	—	0	1	1
—	→	0	1	1

BSF ranges from -1.0 to 1.0, with a value of 0.0 assigned to both the empty graph and fully connected graph. It incorporates all confusion matrix counts, including True Negatives (TNs), thereby considering the relative difficulty of identifying both the presence and absence of edges. The BSF is defined as:

$$BSF = 0.5 \times \left(\frac{TP}{|E|} + \frac{TN}{|M|} - \frac{FP}{|M|} - \frac{FN}{|E|} \right) \quad (6)$$

where $|E|$ and $|M|$ represent the number of edges present and the number of edges absent (with reference to the complete graph) in the true graph respectively, so that:

$$|M| = 0.5 \times n \times (n - 1) - |E| \quad (7)$$

To assess the stability of algorithms, each combination of sample size and network is repeated 25 times with the variable order, variable names, and row order in the dataset all randomised. The normalised BIC score and structural metrics are computed for each of these 25 experiments, and the standard deviation (SD) of the value reported as an indicator of the sensitivity of the algorithm to these randomised dataset artifacts. For each of these 25 experiments, a separate instance of unseen test data is generated, so the reported normalised log-likelihood reflects how well the learned graph generalises across different test samples. Given the large number of individual experiments involved, a time limit of three hours is imposed for structure learning for both categorical and continuous networks.

Taken together, the log-likelihood, structural metrics, and BIC score, along with their respective standard deviations, provide a comprehensive view of performance: how well the learned graph fits the data, how closely it matches the true (data-generating) graph, how effectively each score-based algorithm maximises its objective, and how all these aspects, including their variability, are influenced by dataset artifacts.

5. Results

This section presents the evidence supporting the design decisions for the proposed stable algorithms introduced in Subsection 3.2. The first subsection demonstrates how using a simple, decreasing score, stable order to orientate edges when two arcs have equal scores can make the Tabu algorithm largely insensitive to dataset artifacts. In particular, it includes an ablation study to quantify the contribution of each component of the stable order's sort key described in Subsection 3.2.

While this simple order achieves a high degree of stability, it reduces the structural and data-fitting performance of Tabu. The second subsection shows how the second part of our approach, trying several orders and selecting the one that yields the highest-scoring learned graph, preserves stability while improving performance. The third subsection investigates residual instability when learning categorical networks, and the fourth compares the stability and performance of the proposed algorithms with other well-known approaches.

The first four subsections isolate the impact of dataset artifacts (variable ordering, naming, and row order) by randomising these aspects. The final subsection additionally randomises the sample of rows used at each sample size and for each network, showing that the proposed algorithms offer benefits under this broader form of variation as well.

5.1. Using a decreasing score stable order

We begin by investigating the effectiveness of using a stable order to improve the stability of graphs learned by the Tabu algorithm. In particular, we examine the individual and combined contribution of the three components of the stable order sort key to graph stability. Table 4 reports the mean and SD of the F1 metric, normalised BIC score and normalised log-likelihood, averaged across all categorical networks, sample sizes and dataset randomisations. The first row presents results for the standard unstable Tabu algorithm, that is, where no stable order is used. The remaining three rows show results when a stable order is applied, constructed using one, two or all three of the variable properties described in Subsection 3.2.

Variables are first ordered solely in decreasing order of their score without parents. We observe that this simple stable order already reduces instability considerably: the standard deviation of the F1 score drops by more than a factor of ten, and the SD of

Table 4

The effect of using a stable order, and the number of elements in the stable order key, on mean and SD values of F1, normalised BIC and normalised log-likelihood (LL) for **categorical** networks learned by the Tabu algorithm. Best values are shown in bold, and worst values in bold red text.

Properties used to sort variables in stable order	F1		Normalised BIC		Normalised LL	
	Mean	SD	Mean	SD	Mean	SD
Without stable order	0.443	0.083	-25.4818	0.0445	-23.4432	0.1899
Score with no parents	0.427	0.007	-25.4845	0.0003	-23.4528	0.1504
... <i>plus</i> scores with a parent	0.427	0.007	-25.4844	0.0000	-23.4524	0.1504
... <i>plus</i> counts of values	0.424	0.004	-25.4844	0.0000	-23.4510	0.1502

Table 5

The effect of using a stable order, and the number of elements in the stable order key, on mean and SD values of F1, normalised BIC and normalised log-likelihood (LL) for **continuous** networks learned by the Tabu algorithm. Best values are shown in bold, and worst values in bold red text.

Properties used to sort variables in stable order	F1		Normalised BIC		Normalised LL	
	Mean	SD	Mean	SD	Mean	SD
Without stable order	0.562	0.100	-52.3305	0.0151	-51.5208	0.1583
Score with no parents	0.550	0.000	-52.3299	0.0000	-51.5132	0.1539
... <i>plus</i> scores with a parent	0.550	0.000	-52.3299	0.0000	-51.5132	0.1539

BIC is reduced by over two orders of magnitude. When we additionally sort variables by their conditional scores with one parent (as shown in the third row), the instability in BIC is reduced to 0.0000. Finally, when we also sort using the textual rendition of the value counts (as shown in the last row), the F1 standard deviation is further reduced by a factor of two. The log-likelihood SD is reduced by around one sixth when using only the score with no parents. However, unlike the BIC SD, it does not fall to zero. This is because the log-likelihood is computed on a different randomised test dataset for each experiment, introducing additional variability. For comparison, when log-likelihood is computed on the training data, its SD falls as quickly as that of the BIC. These results demonstrate that each element of the stable sort key proposed in Subsection 3.2 contributes to improving stability when learning categorical networks. We note, however, that the F1 standard deviation remains non-zero even when all three elements are used; we return to this residual instability in Subsection 5.3.

Table 5 shows the analogous results for continuous networks. Note that sorting variables by value counts is not applicable to continuous variables, and so this approach is not included in this table. For continuous variables, applying a stable order based on each variable's score without parents is sufficient to eliminate instability entirely: the standard deviations of the F1 score and BIC reduce to zero. This is presumably because it is extremely unlikely for two continuous variables to share an identical set of values, and therefore unlikely for them to have identical score values.

The results in this subsection demonstrate that using a stable order to orientate edges where both orientations gives the same score improvement completely eliminates instability for continuous variables, and substantially reduces it for categorical variables. However, looking at Tables 4 and 5, we see that this comes at the cost of worsening the F1 performance for categorical and continuous networks, and reducing BIC and log-likelihood for categorical networks. This trade-off motivates the second phase of our proposed approach where the algorithm explores *different stable orders* and uses the one which produces the highest-scoring learnt graph. We investigate this in the next subsection.

5.2. Tabu-Stable and HC-Stable

This subsection investigates the effects of using *different* stable orderings on the BIC score, log-likelihood, and structural metrics of the learned graphs, including their sensitivity to dataset artifacts. We compare the standard, unstable Tabu algorithm with the approach using the three stable orderings discussed in Subsection 3.2: a) using an increasing score order always, or b) a decreasing score order always, or c) incorporating Algorithm 2 which tries both the increasing and decreasing score order and uses the one where a Tabu “lookahead” search produces the highest-scoring learned graph. This last approach is our proposed new algorithm, Tabu-Stable. We also present results for the standard, unstable HC algorithm, and HC-Stable. The latter similarly incorporates Algorithm 2 to determine a stable order, but only allows positive score changes in both the “lookahead” searches to find the best order and in the final hill-climbing phase.

5.2.1. Categorical networks

Table 6 shows mean values of Precision, Recall, F1, BSF, and normalised BIC and log-likelihood averaged across all networks, sample sizes and dataset randomisations. It also presents the SD of F1 and normalised log-likelihood and BIC across the dataset randomisations, averaged across networks and sample sizes.

The stable ordering approaches in both HC and Tabu reduce mean F1 SD by around 30 and 25 times respectively, indicating that using a stable order improves the structural stability of the learned graph considerably. In the following subsections, we demonstrate that the remaining instability arises from duplicate variables. Tabu with a decreasing score order worsens the mean F1 value by 0.019

Table 6

Mean values of structural metrics, normalised BIC and log-likelihood, and stability metrics averaged across all **categorical** networks and sample sizes for standard and stable variants of Tabu and HC. Best values are shown in bold, and worst values in bold red text.

Metric	Tabu				HC	
	Standard (unstable)	Decreasing score order	Increasing score order	Tabu-stable	Standard (unstable)	HC-Stable
Precision	0.494	0.474	0.533	0.555	0.428	0.503
Recall	0.417	0.399	0.449	0.464	0.366	0.425
F1	0.443	0.424	0.478	0.497	0.384	0.451
F1 SD	0.083	0.004	0.003	0.004	0.093	0.003
BSF	0.523	0.507	0.546	0.555	0.492	0.535
Normalised BIC	-25.4818	-25.4844	-25.4849	-25.4576	-25.5031	-25.4684
Norm. BIC SD	0.0445	0.0000	0.0000	0.0000	0.0482	0.0000
Normalised log-lik.	-23.4432	-23.4510	-23.4611	-23.4130	-23.4709	-23.4316
Norm. log-lik. SD	0.1899	0.1502	0.1504	0.1497	0.1919	0.1500

whereas using an increasing score order improves it by 0.035 over Tabu using variable order. However, Tabu-Stable produces the largest improvement in F1 of 0.054 over Tabu, or about 12.2%, as well as offering the best Precision and Recall values. This suggests that choosing the better of the decreasing and increasing score orders improves overall accuracy with categorical networks. HC using variable order is more unstable than Tabu, but HC-Stable reduces this instability considerably and increases mean F1 by 0.0667, around 17%, over standard HC.

These accuracy improvements for categorical networks are also apparent using the alternative BSF metric. HC-Stable improves BSF by around 9% over HC, and Tabu-Stable by approximately 6% over Tabu. Tabu-Stable retains most of the accuracy improvement over HC-Stable that Tabu has over HC, suggesting that the accuracy improvement due to Tabu and that using a stable node order are additive in Tabu-Stable. These results show the improved structural accuracy of using Algorithm 2 to determine a stable order for categorical networks.

Fig. 2 shows the F1 achieved by Tabu with the different orderings for each network at each sample size, with the shaded area around the lines indicating the SD of F1 values at each sample size. This instability is most pronounced for standard Tabu, shown in green, being considerable for most networks. The instability is substantially reduced with all three score-based orderings, although some instability is visible for the Hailfinder network which will be discussed in Subsection 5.3.

The penultimate pair of rows in Table 6 shows the normalised BIC score averaged across all networks and sample sizes. It shows that using a decreasing or increasing score order in all cases produces a lower score than the standard, unstable algorithm, but Tabu-Stable and HC-Stable which use Algorithm 2 both improve the BIC score. The final pair of rows in the table show the mean and SD of normalised log-likelihood for the different orders. As with BIC, using either the decreasing or increasing score stable order in all cases reduced the average log-likelihood, whereas Tabu-Stable and HC-Stable both improved the log-likelihood over their unstable counterparts.

Table 6 also shows that the SD of the normalised BIC score is zero for the approaches using a stable order but non-zero for the standard HC and Tabu algorithms. Thus, the approaches discussed in Subsection 3.2 completely eliminate the instability in the BIC score. The stable approaches reduce log-likelihood SD substantially, but it remains non-zero due to the inherent randomness of the unseen test data used in each experiment. Although we do not present these results, log-likelihood computed on the training data is reduced to zero by the stable algorithms, consistent with the BIC results.

Appendix A provides a breakdown of the F1 metric, BIC and log-likelihood values obtained for each categorical network. These results show that Tabu Stable achieves a higher F1 value than standard Tabu for 13/17 networks, a higher BIC score in 16/17 and a higher log-likelihood in 14/17 networks. Similarly, HC-Stable outperforms standard HC in 14/17 for F1, 16/17 for BIC and 12/17 for log-likelihood. In addition, Table A.16 shows that the **true graph** receives a lower BIC score than all the learnt graphs across all networks highlighting the difficulty of learning the true graph using score-based algorithms using BIC, particularly at low sample sizes.

5.2.2. Performance of continuous networks

Table 7 provides the analogous results to Table 6 but for the seven continuous variable networks. For these networks we again see that the stable order based approaches completely eliminate the instability in the BIC score, log-likelihood based on training data, **and** F1. As with the categorical networks, Tabu-Stable again produces the best normalised BIC score and log-likelihood averaged over all networks. However, the stable order approaches **worsen** F1 for continuous variable networks, with Tabu-Stable achieving a mean F1 of 0.559, compared to 0.562 for standard Tabu, a small decrease of around 0.5%. The F1 for HC-Stable is slightly better than that for HC. The BSF metric offers more favourable results; Tabu-Stable increases BSF by around 1.6% over Tabu, and that for HC-Stable is around 1.5% higher than HC.

Appendix B provides a breakdown of the F1, BIC and log-likelihood values for each continuous network. These results show that Tabu-Stable outperforms standard Tabu in 5/7 networks for F1, all 7 networks for BIC, and 5/7 networks for log-likelihood. Similarly, HC-Stable outperforms standard HC in 5/7 networks for F1, all networks for BIC, but only 2/7 for log-likelihood. The graphs learned by the stable algorithms are generally sparser than those learned by their unstable counterparts, with an average of four fewer edges. This sparsity may partly explain why HC-Stable produces graphs with lower log-likelihood than standard HC.

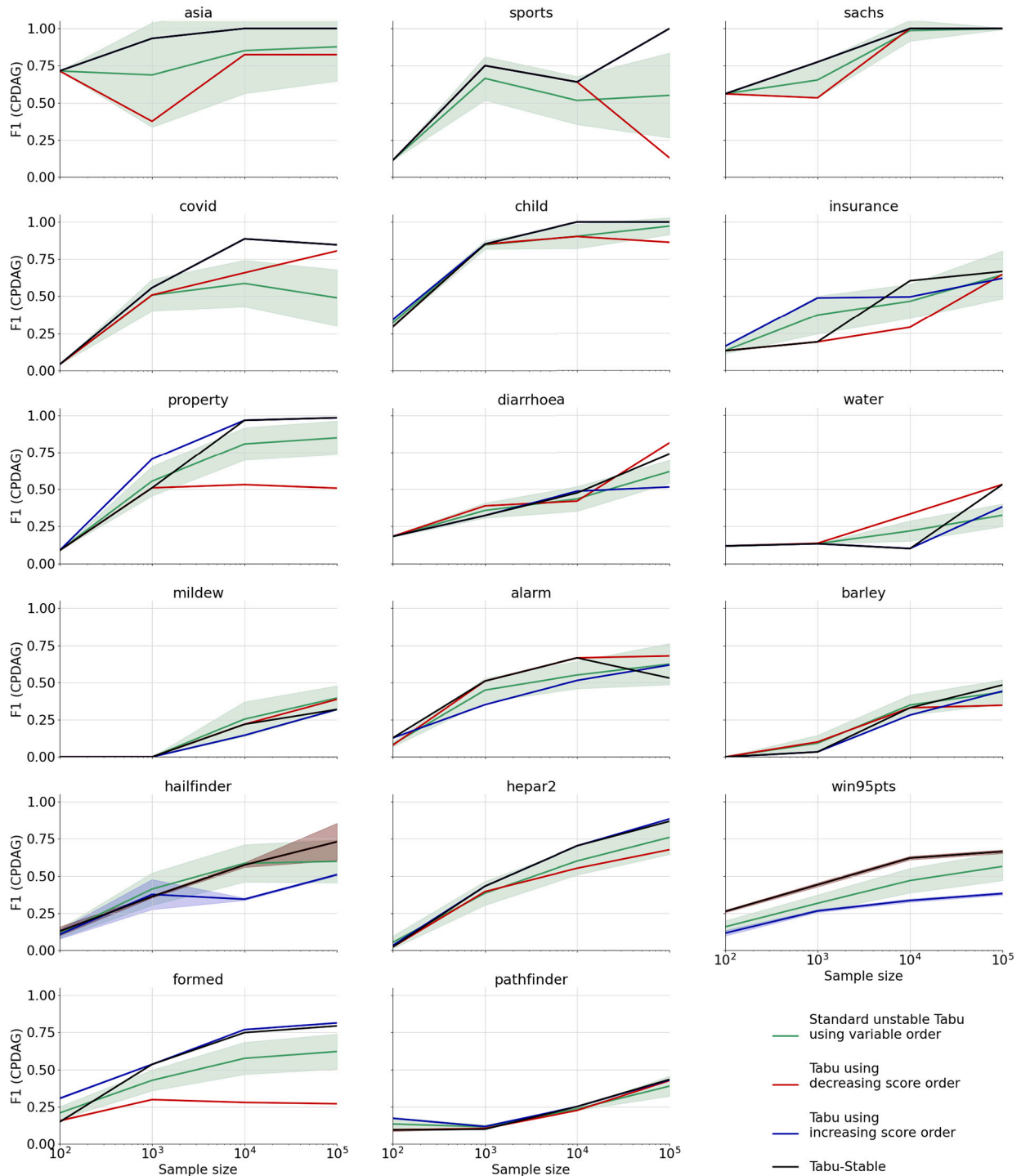


Fig. 2. A comparison of the F1 CPDAG of the categorical variable graphs learned by Tabu using different node orderings: a) variable order, b) simple increasing or c) decreasing score order, and d) Tabu-Stable. 25 experiments with randomised variable names, variable order and row order are conducted for each of the four sample sizes for each network. Shading around lines indicates the SD of F1 values. Note that lines are drawn on the chart in the order shown in the key, so a particular line may be hidden where values are coincident.

5.2.3. Exploring other stable orders

Subsections 5.1 and 5.2 demonstrate that Tabu-Stable and HC-Stable can eliminate instability while also improving BIC, log-likelihood, and F1 scores. However, for continuous networks, structural improvements with Tabu-Stable are mixed: F1 was slightly lower than for standard Tabu, while the alternative BSF metric was slightly higher. This raises the question of whether alternative

Table 7

Mean values of structural, normalised BIC and log-likelihood, and stability metrics averaged across all **continuous** networks and sample sizes for standard and stable variants of Tabu and HC. Best values are shown in bold, and worst values in bold red text.

Metric	Tabu				HC	
	Standard (unstable)	Decreasing score order	Increasing score order	Tabu-stable	Standard (unstable)	HC-Stable
Precision	0.538	0.519	0.453	0.542	0.503	0.510
Recall	0.599	0.595	0.514	0.588	0.574	0.570
F1	0.562	0.550	0.476	0.559	0.530	0.532
F1 SD	0.100	0.000	0.000	0.000	0.100	0.000
BSF	0.753	0.760	0.722	0.766	0.729	0.742
Normalised BIC	-52.3305	-52.3299	-52.3338	-52.3205	-52.3368	-52.3268
Norm. BIC SD	0.0151	0.0000	0.0000	0.0000	0.0160	0.0000
Normalised log-lik.	-51.5208	-51.5132	-51.5249	-51.5118	-51.5252	-51.5194
Norm. log-lik. SD	0.1583	0.1539	0.1545	0.1535	0.1588	0.1548

Table 8

Mean values of structural, normalised BIC and log-likelihood, and stability values averaged across all **categorical** networks and sample sizes for standard Tabu and Tabu using different stable orders. Best values are shown in bold, and worst values in bold red text.

Metric	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt
Precision	0.494	0.555	0.558	0.576
Recall	0.417	0.464	0.467	0.480
F1	0.443	0.497	0.499	0.515
F1 SD	0.083	0.004	0.004	0.004
BSF	0.523	0.555	0.559	0.569
Normalised BIC	-25.4818	-25.4576	-25.4299	-25.3980
Norm. BIC SD	0.0445	0.0000	0.0000	0.0000
Normalised log-lik.	-23.4432	-23.4130	-23.3838	-23.3395
Norm. log-lik. SD	0.1899	0.1497	0.1496	0.1487

Table 9

Mean values of structural, normalised BIC and log-likelihood, and stability values averaged across all **continuous** networks and sample sizes for standard Tabu and Tabu using different stable orders. Best values are shown in bold, and worst values in bold red text.

Metric	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt
Precision	0.538	0.542	0.605	0.626
Recall	0.599	0.588	0.640	0.652
F1	0.562	0.559	0.617	0.633
F1 SD	0.100	0.000	0.000	0.001
BSF	0.753	0.766	0.783	0.790
Normalised BIC	-52.3305	-52.3205	-52.3213	-52.2998
Norm. BIC SD	0.0151	0.0000	0.0000	0.0000
Normalised log-lik.	-51.5208	-51.5118	-51.5116	-51.4999
Norm. log-lik. SD	0.1583	0.1535	0.1535	0.1529

stable orders might yield better performance. To investigate this, we evaluate Tabu using two additional stable orders beyond the one described in Subsection 3.2, where both the decreasing and increasing score orders are tried, and the better-scoring result is retained.

The first new order we introduce is called *Tabu-Stable6*. It builds on the original Tabu-Stable approach but considers six different stable orders instead of just two. Starting from the list of variables in decreasing score order, we divide it into two halves and apply two distinct permutation operations. In the first, we swap the positions of the two halves while preserving the internal order within each half. In the second, we keep the halves in place but reverse the order of variables within each half. These transformations are intended to invert the relative order of many variable pairs. We repeat this process using the increasing score order, resulting in a total of six stable orders: the original decreasing score order, its reverse, and four additional variations obtained by reordering the halves.

Tabu-StableAdapt is the second new stable order we investigate. As with the original HC- and Tabu-Stable approaches, it begins by determining whether the decreasing or increasing score order yields the higher-scoring graph. However, this chosen order is used only as an **initial** stable order, which the algorithm then **adaptively modifies** during the search process. Specifically, whenever hill-climbing encounters a candidate edge addition for which both possible orientations yield the same score increase, it temporarily explores both options by running hill-climbing to completion from each orientation. The orientation that results in the higher final score is then adopted, overriding the initial stable order for that edge. Once the preferred orientation is selected, hill-climbing resumes from that point and continues until the next such ambiguous edge is encountered, at which point the process is repeated. The algorithm terminates once all ambiguous orientations have been resolved in this way.

Table 10

F1 SD over 25 random orderings at different sample sizes using Tabu-Stable for the four networks with residual sensitivity to variable order, and for modified versions of Hailfinder and Win95pts where identical (or duplicate) variables are prevented.

Sample size	hailfinder	win95pts	formed	pathfinder	hailfinder2	win95pts2
10^2	0.030	0.008	0.005	0.009	0.000	0.008
10^3	0.008	0.014	0.000	0.000	0.000	0.000
10^4	0.016	0.012	0.000	0.000	0.000	0.000
10^5	0.125	0.012	0.000	0.000	0.000	0.000

Table 8 presents the results for these new orders when learning categorical networks. All the stable orders have normalised BIC SD of zero, and the F1 SD is greatly reduced compared to standard Tabu showing that Tabu-Stable6 and Tabu-StableAdapt retain the stable behaviour of Tabu-Stable. The structural metrics, BIC and log-likelihood of all three stable approaches are better than those of standard Tabu, those of Tabu-Stable6 are better than Tabu-Stable, and those of Tabu-StableAdapt best of all. F1 for Tabu-StableAdapt is 3.6% better than that of Tabu-Stable, and BSF is 2.5% better. The corresponding results for continuous networks are shown in Table 9. Whereas Tabu-Stable has a lower F1 value for continuous networks than Tabu, the F1 for Tabu-Stable6 and Tabu-StableAdapt are substantially better. As with categorical networks, all the stable approaches had higher BIC scores and log-likelihoods than standard Tabu. These results suggest that the stable orders introduced in this subsection improve performance still further, and that it would be worthwhile investigating other stable orders to see if further improvements can be achieved.

5.3. Analysis of the residual instability in categorical networks with Tabu-Stable

Tabu-Stable returns a F1 SD of 0.000 at all sample sizes for all 7 continuous networks and for 13 out of the 17 categorical networks. Table 10 provides a breakdown of the F1 SD by sample size using Tabu-Stable for the four categorical networks where some structural instability remains. Formed and Pathfinder only have residual instability at a sample size of 100. However, Hailfinder and Win95pts retain some instability at all sample sizes. This is because both networks have some local structures and CPT values that deterministically create identical values for pairs of variables at all sample sizes. The last two columns in Table 10 show results for versions of these networks that are modified slightly to remove these deterministic relationships. Hailfinder2 and Win95pts2 have had one node and two arcs removed respectively to achieve this. Table 10 shows that instability has been removed completely for Hailfinder2 and only remains at the smallest sample size for Win95pts2 because some variables are identical.

5.4. Comparing Tabu-Stable and HC-Stable with other algorithms

5.4.1. Categorical networks

Fig. 3 compares the mean structural, normalised log-likelihood and BIC score and stability metrics for the categorical networks achieved by different algorithms with those obtained by HC-Stable, Tabu-Stable and Tabu-StableAdapt. The implementation of FGES from the Tetrad package [41], and MMHC, H2PC, PC-Stable, GS and Inter-IAMB from the bnlearn package [40] are used. Results for standard Tabu and HC using variable order are also included. The results use the variants of Hailfinder and Win95pts discussed in Subsection 5.3 which avoid identical variables.

Additionally, the bnlearn algorithms reject datasets which contain any variable that has the same value for all rows, so the cases where this occurs are also excluded from this comparison. These are the datasets with 100 rows for the Insurance, Water, Barley, Hailfinder, Win95pts, Formed and Pathfinder networks. Excluding experiments where datasets contain identical or single-valued variables means that the algorithms have potential to achieve full stability, provided they are *truly* stable. Thus, all algorithms are compared across the same experiments, with the exception that FGES failed to complete within 3 hours for sample sizes of 10^4 and 10^5 for Hailfinder and Pathfinder, that is, for 4/58 experiments. We ignore these failure cases when computing the results shown in Fig. 3. This introduces some bias into the results but, as we show below, when we impute missing results instead, this has a relatively small effect and did not change the ranking of algorithms. Moreover, result imputation introduces its own biases; for example, making FGES appear more unstable than it actually is.

Tabu-Stable, Tabu-StableAdapt and HC-Stable are the only algorithms to produce a mean F1 SD of zero, by completely eliminating structural instability. FGES is found to be *almost* structurally stable with a mean F1 SD of 0.002. The other algorithms, including PC-Stable which has a mean F1 SD of 0.037, all exhibit considerable structural instability. Moreover, Tabu-StableAdapt offers the highest mean F1 of 0.555, with Tabu-Stable second at 0.536. Note that these values are higher than those quoted in Table 6 because the datasets not considered in this set of experiments, due to duplicate or single-valued variables, tend to be those with lowest sample sizes that also tend to produce lower F1 values. FGES produces the third highest mean F1 of 0.528 when its failure cases are ignored, or 0.519 if its failure cases are assigned the mean F1 value obtained across all the other algorithms. Tabu-StableAdapt has the best precision (0.614), with FGES next (0.605) and then Tabu-Stable (0.590). Tabu-StableAdapt also has the best recall (0.521), followed by Tabu-Stable (0.505) and then FGES (0.487).

Tabu-StableAdapt had the best log-likelihood (-22.8754) and BIC (-24.4251), with other Tabu and HC variants coming close, and FGES sixth, on both measures. The hybrid and constraint-based algorithms obtained considerably worse log-likelihood and BIC scores which is expected since they are not designed to just maximise an objective score. As noted previously, Tabu-StableAdapt, Tabu-Stable and HC-Stable achieved zero BIC SD, that is, complete BIC stability too. FGES had very stable BIC scores with SD of

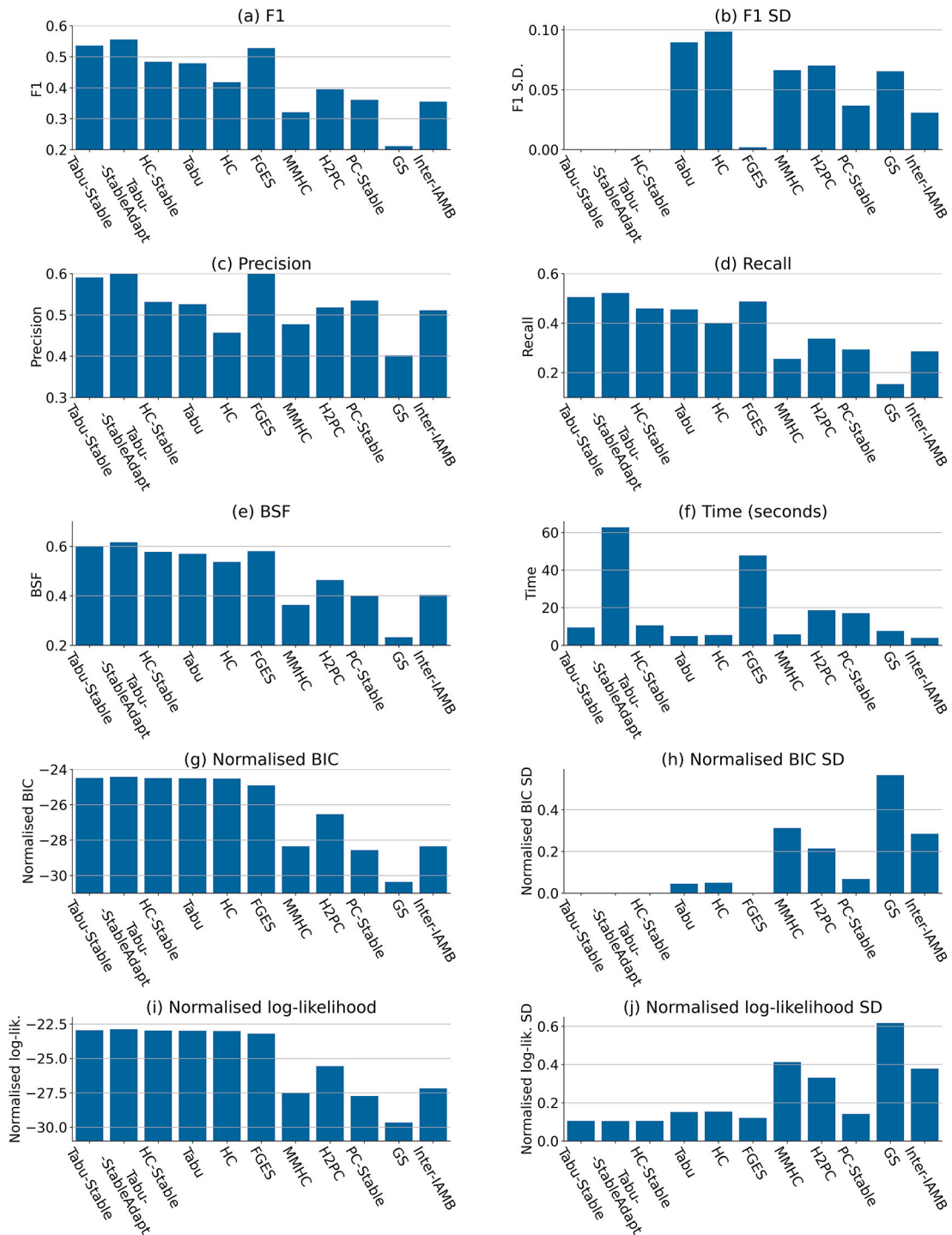


Fig. 3. Mean values of structural, inference and stability metrics across categorical networks and sample sizes for different algorithms.

0.0018. Standard Tabu, HC and PC-Stable had a moderate BIC SD below 0.1. GS, Inter-IAMB, MMHC and H2PC demonstrated much larger log-likelihood and BIC instability.

Tabu-Stable and HC-Stable were nearly twice as slow as their unstable counterparts, but were generally competitive with the runtimes of the other algorithms; for example, Tabu-Stable was around twice as fast as PC-Stable and H2PC, and much faster than FGES. FGES was the second slowest algorithm tested with a mean run time around 50 seconds, but most of this larger mean execution time was due to extended runtimes on a few networks, notably Mildew and Hailfinder2. Tabu-Stable is faster than HC-Stable, and Tabu is faster than HC, which is surprising since the Tabu approach generally performs more iterations than plain HC. Tabu may be

Table 11

Networks and sample sizes at which algorithms failed to learn a graph within the 3 hour time limit. Combinations where all dataset randomisations failed to learn a graph are marked with a red cross, and those where only some of the randomised datasets failed are marked with a blue cross.

Network	Sample Size	FGES	MMHC	H2PC	PC-Stable	GS	Inter-IAMB
Magic-NIAB	100,000			✗		✗	✗
Magic-IRRI	100,000	✗		✗		✗	✗
Ecoli70	100,000	✗					
Arth150	10,000	✗		✗		✗	
Arth150	100,000	✗	✗	✗	✗	✗	✗

Table 12

Percentage of learned graphs that were non-extendable, and therefore could not be allocated an objective score.

Network	PC-Stable	GS	Inter-IAMB
Categorical	12.9	2.4	16.4
Continuous	28.3	67.6	68.0

faster because its *tabulist* prevents it from considering some recently visited high-scoring graphs and thereby avoids relatively costly acyclicity checks for those graphs. Tabu-StableAdapt was the slowest algorithm with a mean runtime of around 60 seconds due to its repeated modification and evaluation of the stable order. It is nonetheless not much slower, on average, than FGES.

5.4.2. Continuous networks

Fig. 4 presents equivalent algorithm comparisons for the seven continuous networks. Once again, we show averages across all sample sizes and networks, but ignore individual failure cases for a particular algorithm, sample size and network; these failure cases are discussed further below. Tabu-Stable, Tabu-StableAdapt, HC-Stable and FGES all have F1 and BIC SD values of 0.0 demonstrating complete structural and score stability with these continuous networks. The F1 SD for Inter-IAMB is modest at 0.008, but the other algorithms have F1 SD values between 0.029 and 0.099 indicating considerable structural instability.

Standard Tabu and HC have relatively low values for normalised BIC SD of 0.0168 and 0.0178 respectively. Except for FGES, the other algorithms exhibit much larger log-likelihood and BIC instability. It is perhaps to be expected that the score-based algorithms have the more stable BIC scores since they aim to maximise the objective score. Tabu-StableAdapt achieves the best normalised log-likelihood and BIC score, with the other Tabu and HC algorithms close behind, and FGES with the sixth best log-likelihood and BIC score some way behind. The hybrid and constraint-based algorithms have considerably worse log-likelihood and BIC scores.

The networks and sample sizes where experiments failed to complete within the time limit are shown in Table 11. In some cases, runtimes were far in excess of three hours for a particular sample size and network, so that none of the randomised variations produced a result within the time limit which is shown by a red cross in Table 11. In other cases, marked by a blue cross in the table, some of the randomised datasets produced a result within the time limit and others did not. Thus, the execution runtime can also be significantly affected by dataset artifacts such as variable ordering in the algorithms shown in Table 11.

Another complication when comparing the algorithms is that constraint-based algorithms often produce a PDAG which cannot be extended into a DAG, and therefore cannot be represented as a CPDAG nor assigned an objective score. Table 12 shows the percentage of runs that completed within the time limit but which produced a PDAG that could not be extended. Note that, in general, this failure to learn extendable graphs occurred across most networks and sample sizes. Typically, some of the learned graphs for a particular network and sample size are extendable and others not, so this represents yet another feature of the learned graph that is sensitive to dataset artifacts. Inter-IAMB is particularly prone to producing non-extendable PDAGs and did not produce *any* extendable PDAGs for the Covid, Magic-NIAB, Magic-IRRI and Arth150 continuous networks. This means that the mean log-likelihood and BIC score for Inter-IAMB for continuous networks is not comparable with the other algorithms, and hence Inter-IAMB is not shown in Figs. 4(g) and 4(i).

Unlike the results for the categorical networks, the better log-likelihood and BIC scores obtained by the stable compared to standard Tabu and HC are not always reflected by better structural accuracy. The F1 for Tabu-Stable is 0.554 is only very slightly better than that achieved by standard Tabu, 0.552. However, the F1 for Tabu-StableAdapt is considerably higher at 0.619, second only behind FGES at 0.685. HC-Stable achieves 0.522, 0.001 worse than standard HC. H2PC has a similar F1 value to HC and HC-Stable, with the other hybrid and constraint-based algorithms close behind, except for PC-Stable which is considerably worse. If we impute F1 values for failed experiments then the mean F1 values for H2PC, GS and Inter-IAMB rise by around 0.02 to 0.03 raising them above HC-Stable but not altering the picture dramatically. Tabu-Stable and Tabu-StableAdapt fared better overall according to the BSF structural metric, where they had the highest values of 0.784 and 0.763 compared to FGES at 0.732. The hybrid and constraint-based algorithms are substantially less accurate than the score-based algorithms according to the BSF metric.

The BIC objective score seems to be a better indicator of correct structure when used with categorical data than it is with continuous data, a finding also reported in [34] and [35]. Table 13 suggests a reason why this might be so. It shows the mean of node degree and mean of node degree SD of the learned graph for each algorithm for the categorical and continuous networks, where node degree is the sum of in-degree and out-degree for a node. The node degree reflects the density of the graph skeleton, and the node degree

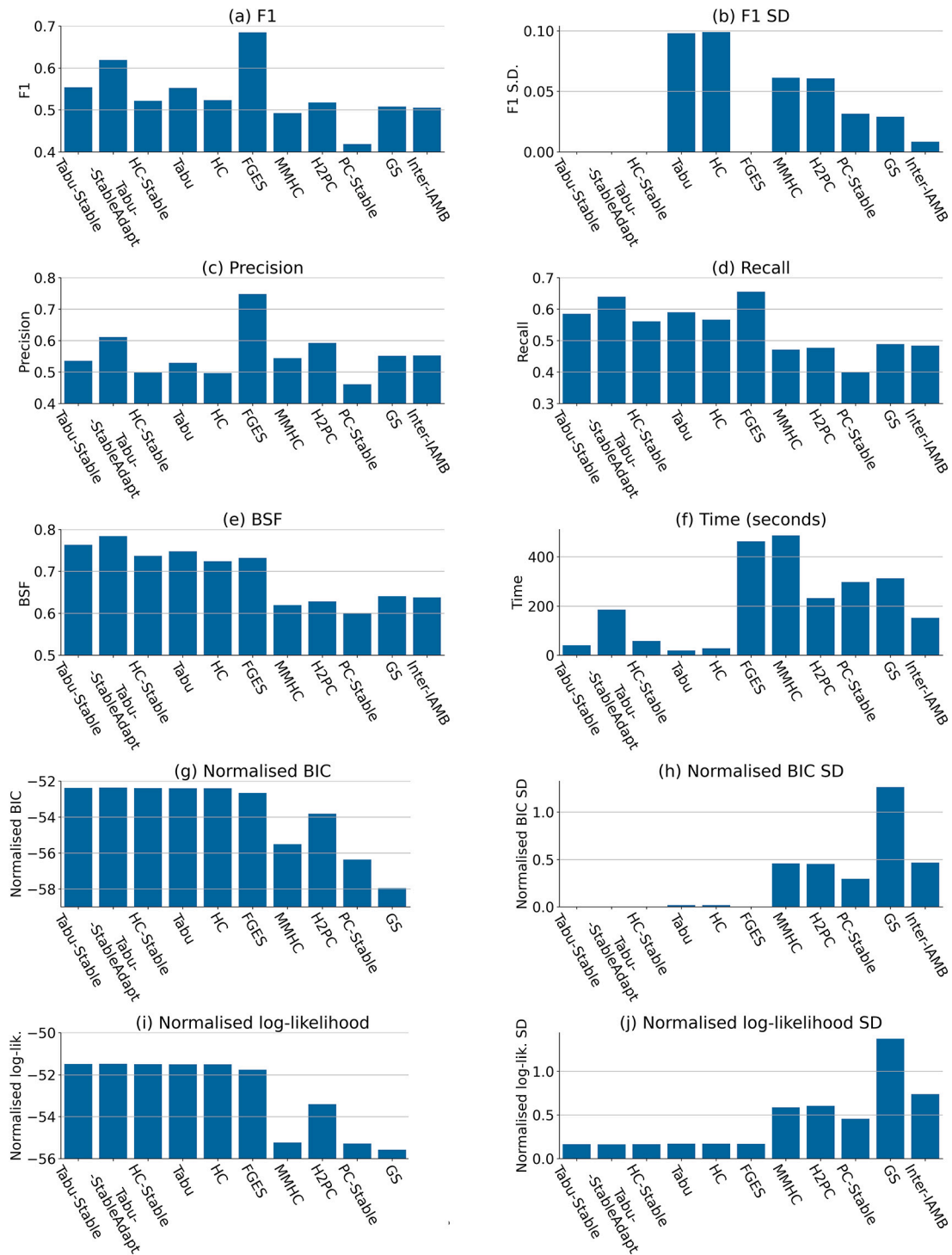


Fig. 4. Mean values of structural, inference and stability metrics across **continuous** networks and sample sizes for different algorithms.

SD the instability of the skeleton. The mean node degree of the true graphs is also shown: 3.2328 for the categorical networks and 3.0288 for the continuous networks. With the categorical networks, all algorithms tend to learn graphs with a **lower** node degree, that is, density, than the true graphs, but the HC/Tabu family of algorithms learn graphs with a density closest to the true value, contributing to their good structural metrics. Conversely, all the algorithms learn graphs with a **higher** mean density than the true mean density for the continuous networks. Once again, the HC/Tabu family of algorithms learn the densest graphs, but this now means they are furthest away from the true density, contributing to poorer structural metrics.

Table 13

Mean of node degree and mean of node degree SD for categorical and continuous networks for the true graph and graphs learned by the different algorithms.

Algorithm	Categorical networks		Continuous networks	
	Mean of node degree	Mean of node degree SD	Mean of node degree	Mean of node degree SD
True graph	3.2328	n/a	3.0288	n/a
HC	2.5603	0.1041	3.7771	0.2478
HC-Stable	2.4917	0.0000	3.6392	0.0000
Tabu	2.5155	0.0882	3.6871	0.2335
Tabu-Stable	2.4655	0.0000	3.5660	0.0000
Tabu-StableAdapt	2.4589	0.0000	3.4669	0.0000
FGES	2.2547	0.0008	2.6061	0.0000
MMHC	1.4307	0.0659	2.6017	0.0242
H2PC	1.7904	0.0512	2.3539	0.0020
PC-Stable	1.4690	0.0000	2.5813	0.0008
GS	1.0265	0.1301	2.7062	0.0600
Inter-IAMB	1.5578	0.0375	2.7082	0.0095

Table 14

Stability and performance of standard (unstable) Tabu, Tabu-Stable and Tabu-StableAdapt for **categorical** networks with and without variable ordering and row sample randomisation. Best values are shown in bold, and worst values in bold red text.

Tabu variant	Randomisation		F1		Normalised BIC		Norm. log-lik.	
	Order	Sample	Mean	SD	Mean	SD	Mean	SD
Standard	No	No	0.497	n/a	-25.1523	n/a	-23.5922	n/a
Standard	Yes	No	0.479	0.090	-24.5091	0.0452	-22.9853	0.1522
Stable	Yes	No	0.536	0.000	-24.4914	0.0000	-22.9578	0.1050
StableAdapt	Yes	No	0.555	0.000	-24.4251	0.0000	-22.8754	0.1046
Standard	No	Yes	0.497	0.051	-24.5557	0.1185	-22.9864	0.1400
Standard	Yes	Yes	0.466	0.102	-24.5647	0.1421	-23.0057	0.1759
Stable	Yes	Yes	0.519	0.053	-24.5370	0.1192	-22.9799	0.1463
StableAdapt	Yes	Yes	0.561	0.054	-24.4798	0.1201	-22.9059	0.1441

Table 13 also shows that GS, Inter-IAMB, MMHC and H2PC may have relatively high values for the density SD, indicating that data artifacts can also affect the skeleton of the learned graph. Interestingly, PC-Stable has low values for the density SD suggesting that the implementation we are using *may* have a stable skeleton determination phase, and the structural instability seen in the F1 SD value stems from arc orientation instability in the v-structure or orientation propagation phases of PC-Stable. However, as we have not inspected the implementation, this interpretation remains speculative.

5.5. The effect of row sampling

The results we have presented so far show that our proposed algorithms are stable in the sense that we have used throughout this paper; that is, for a *given sample* of data rows, the resultant learned graph is not affected by dataset artifacts such as row order, variable names, and in particular, variable order. However, this final set of results places this improvement in context by comparing the gains we have made in stability with our proposed algorithms with the variability in results when graphs are learned from *different samples* taken from the underlying global probability distribution.

Table 14 shows the performance obtained by the Tabu algorithm when learning categorical networks in eight series of experiments, each varying whether the variable order and row samples are randomised. The first column indicates which variant of Tabu is used: the standard unstable version, or one of our two proposed stable algorithms, Tabu-Stable and Tabu-StableAdapt. The second column specifies whether the variable order, variable names, and row order are randomised across the 25 cases run at each sample size and network. The third column shows whether the sample of rows is randomised: if so, a different selection is drawn randomly from one million rows for each of the 25 cases; if not, the same row selection is reused across cases. Performance and stability are summarised using the mean and SD of F1 score, normalised BIC, and log-likelihood. The first row presents results where neither dataset artifacts nor the sample of rows are varied; here, only a single dataset is used at each network and sample size, so SD is not applicable. In the next three rows, dataset artifacts are randomised but the row sample remains fixed; these are the same experiments discussed in the previous subsection. In the final four rows, the row sample is randomised.

Looking at the F1 SD column in Table 14, the highest variability ($SD = 0.102$) occurs when both dataset artifacts and the sample are randomised and the standard Tabu algorithm is used. This variability is nearly halved when using our stable Tabu variants. This remaining variability is expected, as the stable algorithms are now learning from fundamentally different datasets at each sample size and network. The improvement in F1 variability is most pronounced at high sample sizes; if we consider only the 10^5 sample size, then the stable algorithms reduce F1 SD by around four times. This effect of sample size is possibly expected, as sample randomisation may have less impact at higher sample sizes, and so conversely, the reduction in variability due to dataset artifacts has proportionately more impact. The stable variants retain their improved F1 performance over standard Tabu when both order and

Table 15

Stability and performance of standard (unstable) Tabu, Tabu-Stable and Tabu-StableAdapt for **continuous** networks with and without variable ordering and row sample randomisation. Best values are shown in bold, and worst values in bold red text.

Tabu variant	Randomisation		F1		Normalised BIC		Norm. log-lik.	
	Order	Sample	Mean	SD	Mean	SD	Mean	SD
Standard	No	No	0.565	n/a	-52.3208	n/a	-51.5678	n/a
Standard	Yes	No	0.562	0.100	-52.3305	0.0151	-51.5208	0.1583
Stable	Yes	No	0.559	0.000	-52.3205	0.0000	-51.5118	0.1535
StableAdapt	Yes	No	0.633	0.001	-52.2998	0.0000	-51.4999	0.1529
Standard	No	Yes	0.536	0.077	-52.2682	0.1561	-50.9991	0.2165
Standard	Yes	Yes	0.526	0.108	-52.2695	0.1520	-50.9798	0.1653
Stable	Yes	Yes	0.560	0.085	-52.2572	0.1524	-50.9680	0.1647
StableAdapt	Yes	Yes	0.610	0.076	-52.2555	0.1570	-51.2563	0.1588

sample are randomised—Tabu-StableAdapt achieves a mean F1 score nearly 0.1 higher than standard Tabu. These stable variants also maintain superior normalised BIC and log-likelihood scores, and reduced variability in them, when the dataset sample is randomised.

The corresponding results for continuous networks are shown in Table 15. We again observe a reduction in F1 SD with the stable algorithms when row sampling is used. However, the reduction is less pronounced than for categorical networks, falling from an SD of 0.108 for standard Tabu to 0.085 and 0.076 for Tabu-Stable and Tabu-StableAdapt respectively. Again, reductions in F1 SD from the stable algorithms eliminating variability due to dataset artifacts were proportionately greater at larger sample sizes because variability due to sampling noise was lower. Both Tabu-Stable and Tabu-StableAdapt produced improvements in F1 and normalised BIC over standard Tabu when sampling noise is present. Tabu-Stable improved log-likelihood over standard Tabu, but although Tabu-StableAdapt performed better in 6/7 networks, a relatively poor result on *arth150_c* causes its mean log-likelihood across all continuous networks to be lower than that of standard Tabu.

Fig. 5 compares the performance and stability of different algorithms when learning categorical networks where both dataset artifacts and the row sample are randomised. Looking at Fig. 5(b) we see that the stable variants of Tabu reduce F1 SD considerably as noted previously, but we also see that the F1 SD for standard Tabu and HC is high compared to other algorithms suggesting that the standard versions are particularly sensitive to the row sample randomisation. However, the F1 SDs of Tabu-Stable (0.053), HC-Stable (0.050) and Tabu-StableAdapt (0.054) are lower than all other algorithms except for FGES (0.046). Tabu-StableAdapt achieves the highest F1 value (0.561), followed by FGES (0.533) and then Tabu-Stable (0.519). Tabu-Stable and Tabu-StableAdapt have the highest normalised BIC score and log-likelihood. Tabu-Stable and Tabu-StableAdapt have the smallest BIC SD, and the smallest log-likelihood SD together with HC-Stable.

The results for continuous networks are shown in Fig. 6. FGES achieves the highest F1 score (0.630), followed by Tabu-StableAdapt (0.610) and Tabu-Stable (0.560). Standard Tabu and HC exhibit the largest F1 SD values, whereas their stable counterparts have considerably lower F1 SD. However, even the stable versions show higher F1 SD than FGES and the constraint-based algorithms. Despite this, both the standard and stable variants of Tabu and HC achieve the highest and most stable BIC scores and log-likelihoods.

In summary, this subsection shows that Tabu-Stable and Tabu-StableAdapt improve upon standard Tabu in both stability and performance when dataset artifact noise and sampling noise are present, for both categorical and continuous networks. Similarly, HC-Stable outperforms standard HC in this setting. Compared to other score-based, constraint-based, and hybrid algorithms, Tabu-StableAdapt and Tabu-Stable consistently produce the highest and most stable log-likelihood and BIC scores across both data types, with the exception of the log-likelihood for Tabu-StableAdapt for continuous variables which is lower than that for the other HC and Tabu algorithms. Tabu-StableAdapt achieves the highest F1 score for categorical networks, and the second highest for continuous networks (behind FGES), with Tabu-Stable ranking third in both cases. Our stable algorithms have lower F1 SD than all others except FGES for categorical networks, although the F1 SD remains relatively high for continuous networks, possibly due to the pronounced sensitivity of standard HC and Tabu to sampling noise.

6. Concluding remarks

We present HC-Stable and the Tabu-Stable family of hill-climbing algorithms that eliminate instability in learning categorical and continuous variable networks, provided the dataset does not contain variables with identical value sequences. Unlike many well-known algorithms, our approach ensures that learned graphs are insensitive to dataset artifacts such as variable ordering, variable naming and row order.

We show that this instability in standard algorithms can affect nearly every aspect of the learned result, including the objective score, log-likelihood, skeleton, arc orientations, and even runtime. For some constraint-based methods, instability may also prevent conversion of the PDAG into a valid CPDAG or DAG. Our method overcomes these issues by first determining a stable node order based on objective scores, which is then used in subsequent hill-climbing to resolve arc orientations when scores are tied. The approach includes a “lookahead” phase where multiple stable orders are explored, and the one yielding the highest objective score is adopted improving performance whilst retaining stability.

These new algorithms demonstrate clear benefits under both dataset artifact and sampling noise, showing improvements over other approaches:

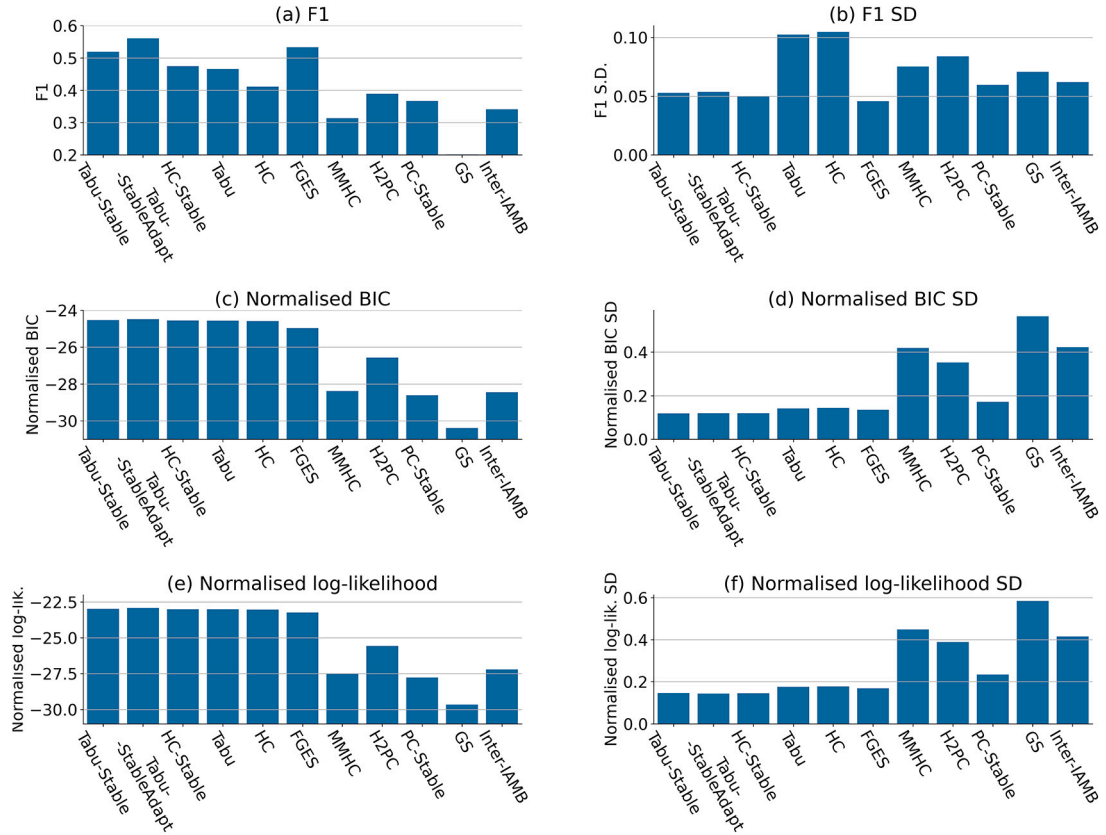


Fig. 5. Mean values of structural, inference and stability metrics across **categorical** networks and sample sizes for different algorithms where rows are sampled randomly.

- **Stability:** HC-Stable, Tabu-Stable, and its variants produce the same results regardless of variable ordering, naming, or row ordering for a given algorithm, network and set of data rows. By eliminating sensitivity to dataset artifacts, they reduce variability in results when both artifact and sampling noise are present.
- **Structural accuracy:** For categorical networks, Tabu-StableAdapt achieves the highest F1 score, followed by FGES and Tabu-Stable. For continuous networks, FGES ranks highest, followed by Tabu-StableAdapt and Tabu-Stable. Under the BSF metric, Tabu-StableAdapt and Tabu-Stable are top performers across both network types.
- **Data fitting and objective score:** Tabu-StableAdapt and Tabu-Stable yield the highest mean normalized log-likelihood and BIC scores across both network types, except for log-likelihood results from Tabu-StableAdapt with continuous data and sampling noise.
- **Efficiency:** Despite additional computations, their runtimes remain competitive, with Tabu-Stable significantly outperforming and Tabu-StableAdapt remaining competitive with FGES, the only other very stable algorithm here, for both categorical and continuous networks.
- **Scalability:** FGES, the constraint-based and hybrid algorithms all produce cases where runtime is unusually high for particular networks whereas the new algorithms proposed in this study demonstrate consistent runtimes across all networks investigated.

For networks with continuous variables, we observe that our proposed algorithms sometimes decrease structural accuracy even when BIC scores improve. This highlights a limitation of score-based algorithms: they are designed to optimise a specific objective score, but this does not always translate into more accurate graphs, particularly when sample sizes are modest. In this study, this issue is evident when the proposed algorithms produce graphs with higher BIC scores than the true graph but the learned graph is structurally inaccurate.

The methodology is flexible and can incorporate alternative strategies for determining a stable variable order. These may include extending the analysis in Subsection 5.2.3 to explore new permutations and evaluations of stable node orders based on variable scores, or using other criteria to order variables [31,30] that may better reflect the true graph. The approach is general and applicable to greedy score-based or hybrid algorithms that include hill-climbing. Additionally, this mechanism could be embedded in the scoring function itself, by adding a small delta to specific edge orientations to reflect the stable order and guide tie-breaking decisions making it

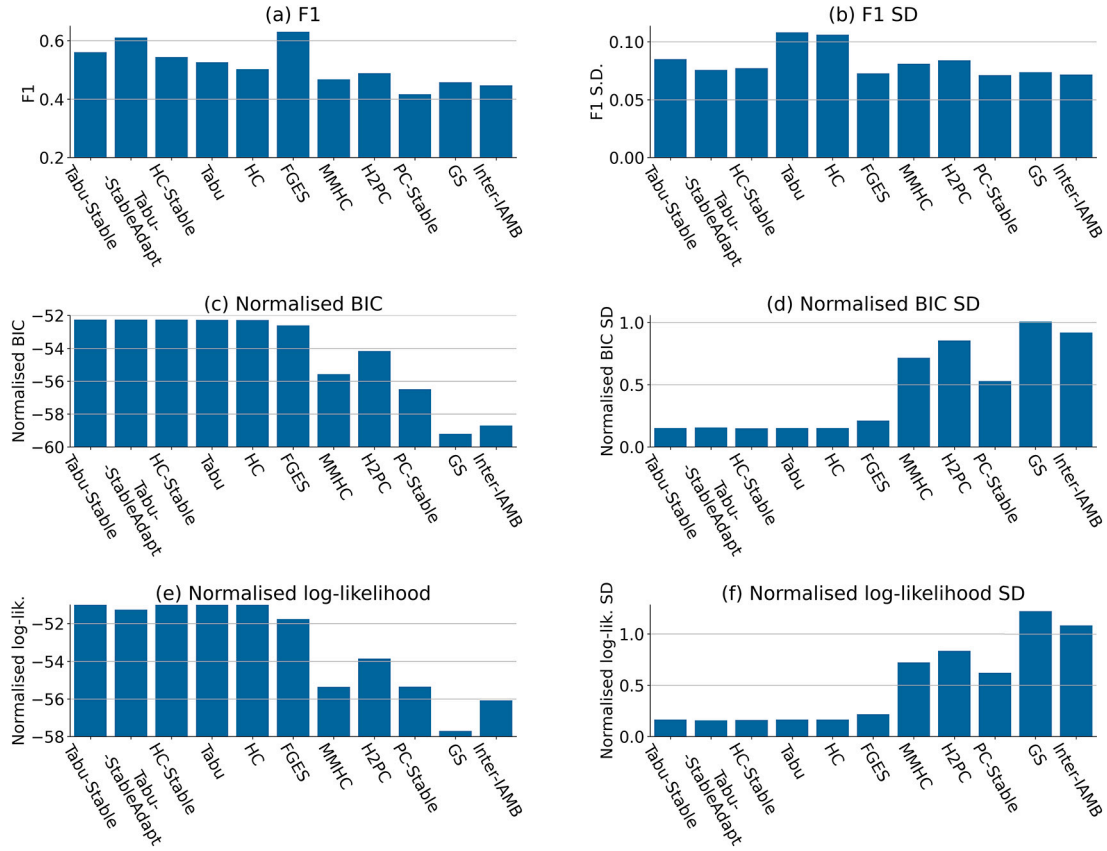


Fig. 6. Mean values of structural, inference and stability metrics across continuous networks and sample sizes for different algorithms where rows are sampled randomly.

unnecessary to modify the algorithms themselves to incorporate our approach. The suggestions in this paragraph all provide potential promising directions for further work.¹

We recommend the Tabu-Stable family of algorithms for practical applications and algorithm comparisons, given its stability, simplicity, and robust performance. The algorithms, along with instructions for their use and for reproducing all results and charts presented here, are freely available on GitHub at <https://github.com/causal-iq/discovery>. Moreover, we emphasise that stability considerations should be integral to algorithm design and evaluation, as instability can bias benchmarks and practical outcomes. Investigating the causes of instability can further inspire the development of new and improved algorithms.

CRedit authorship contribution statement

Neville K. Kitson: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Anthony C. Constantinou:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Performance breakdown by categorical network

Tables A.16, A.17, and A.18 report the F1 score, normalised BIC, and log-likelihood values obtained for each categorical network in the experiments described in Subsection 5.2, where different methods for exploring the stable ordering space are evaluated. The second and third columns of each table show results obtained using the standard (unstable) Tabu algorithm and the Tabu-Stable algorithm, which selects the better of the decreasing and increasing score-based stable orders. Likewise, the sixth and seventh columns show

¹ We thank one of our anonymous reviewers for suggesting the application to a wider range of algorithms, and to embedding the approach in the scoring function.

Table A.16

F1 metric of **categorical** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC	
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable
asia	0.782	0.912	0.912	0.912	0.488	0.597
sports	0.460	0.625	0.625	0.625	0.411	0.653
sachs	0.799	0.833	0.833	0.833	0.739	0.826
covid	0.406	0.491	0.510	0.559	0.366	0.491
child	0.760	0.786	0.786	0.798	0.634	0.602
insurance	0.403	0.398	0.467	0.533	0.357	0.362
property	0.574	0.637	0.603	0.594	0.469	0.637
diarrhoea	0.398	0.429	0.403	0.372	0.404	0.440
water	0.199	0.289	0.279	0.289	0.185	0.226
mildew	0.163	0.148	0.148	0.159	0.151	0.114
alarm	0.428	0.422	0.417	0.512	0.377	0.417
barley	0.220	0.229	0.259	0.321	0.205	0.195
haifinder	0.428	0.449	0.445	0.449	0.351	0.380
hepar2	0.450	0.513	0.528	0.503	0.409	0.461
win95pts	0.378	0.498	0.498	0.465	0.356	0.501
formed	0.458	0.571	0.566	0.599	0.419	0.543
pathfinder	0.218	0.213	0.212	0.221	0.211	0.222

Table A.17

Normalised BIC score of **categorical** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC		Baseline
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable	True graph
asia	-2.3383	-2.3374	-2.3374	-2.3374	-2.3437	-2.3383	-2.3537
sports	-11.7874	-11.7538	-11.7538	-11.7538	-11.8079	-11.7549	-16.9389
sachs	-7.6888	-7.6862	-7.6862	-7.6862	-7.6968	-7.6901	-8.2304
covid	-14.3985	-14.3042	-14.2650	-14.2378	-14.4524	-14.3081	-63.1447
child	-13.0724	-13.0642	-13.0642	-13.0413	-13.1136	-13.0811	-13.3947
insurance	-14.6687	-14.6546	-14.6305	-14.5824	-14.6990	-14.6682	-17.8317
property	-27.6299	-27.6288	-27.6234	-27.6231	-27.6374	-27.6288	-43.2863
diarrhoea	-19.9769	-19.9689	-19.9664	-19.9678	-19.9812	-19.9696	-29.9880
water	-13.6683	-13.6604	-13.6593	-13.6572	-13.6700	-13.6636	-22.8180
mildew	-54.4729	-54.4282	-54.4282	-54.4211	-54.4897	-54.4579	-674.7136
alarm	-12.1061	-12.0830	-12.0641	-12.0119	-12.1311	-12.0977	-13.2097
barley	-65.8108	-65.8961	-65.5395	-65.4097	-65.8643	-65.9064	-562.0477
haifinder	-54.0439	-54.0042	-54.0039	-54.0042	-54.0661	-54.0150	-65.0452
hepar2	-33.7398	-33.7342	-33.7305	-33.7229	-33.7464	-33.7419	-41.8153
win95pts	-10.1453	-10.0883	-10.0883	-10.0713	-10.1489	-10.0899	-11.8806
formed	-43.9545	-43.8664	-43.8633	-43.8104	-43.9722	-43.8835	-46.9654
pathfinder	-33.6885	-33.6200	-33.6042	-33.4278	-33.7318	-33.6686	-254.3176

Table A.18

Log-likelihood of **categorical** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC		Baseline
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable	True graph
asia	-2.2352	-2.2352	-2.2352	-2.2352	-2.2416	-2.2350	-2.2315
sports	-11.1408	-11.1077	-11.1077	-11.1077	-11.1588	-11.1176	-9.9247
sachs	-7.2771	-7.2761	-7.2761	-7.2761	-7.2896	-7.2757	-7.0247
covid	-12.6236	-12.5463	-12.4176	-12.4547	-12.7516	-12.5263	-10.1396
child	-12.2577	-12.2804	-12.2804	-12.2218	-12.2713	-12.2816	-11.8189
insurance	-13.5406	-13.5687	-13.4633	-13.4575	-13.5558	-13.5742	-12.6408
property	-24.6195	-24.6039	-24.6109	-24.6126	-24.6273	-24.6039	-22.7112
diarrhoea	-19.4089	-19.4047	-19.3983	-19.3978	-19.4214	-19.4046	-18.4231
water	-13.0027	-12.9982	-12.9970	-12.9924	-13.0099	-13.0181	-12.6705
mildew	-48.7034	-48.6143	-48.6143	-48.6132	-48.7440	-48.6718	-35.2846

the corresponding results for the standard (unstable) HC algorithm and its stable counterpart, HC-Stable. This appendix therefore provides a per-network breakdown of the aggregate results presented in Table 6 in Subsection 5.2.1.

Table A.18 (continued)

Network	Tabu				HC		Baseline
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable	True graph
alarm	-10.6929	-10.5988	-10.5830	-10.5688	-10.7153	-10.6144	-10.1019
barley	-59.8961	-59.8517	-59.6180	-59.2731	-59.9909	-60.0228	-43.2624
haillfinder	-50.4027	-50.3592	-50.3777	-50.3592	-50.4320	-50.3738	-47.1049
hepar2	-32.4576	-32.4643	-32.4712	-32.4583	-32.4642	-32.4674	-31.6778
win95pts	-9.5469	-9.4913	-9.4913	-9.4861	-9.5494	-9.4942	-8.8217
formed	-42.0301	-41.9346	-41.9316	-41.8929	-42.0441	-41.9402	-40.8860
pathfinder	-28.6987	-28.6856	-28.6510	-28.3640	-28.7386	-28.7155	-20.3950

The fourth and fifth columns in each table present results from Tabu-Stable6 and Tabu-StableAdapt, offering a network-level breakdown of the summary presented in Table 8 in Subsection 5.2.3. Finally, the last columns of Tables A.17 and A.18 show the normalised BIC and log-likelihood values for the true (data-generating) networks, for comparison against the scores of the learned structures.

Appendix B. Performance breakdown by continuous networks

Tables B.19, B.20, and B.21 report the F1 score, normalised BIC, and log-likelihood values obtained for each continuous network in the experiments described in Subsection 5.2, where different methods for exploring the stable ordering space are evaluated. These tables follow the format of those in Appendix A and provide a network-level breakdown of the aggregate results for continuous networks reported in Tables 7 and 9 in Subsections 5.2.2 and 5.2.3 respectively.

Table B.19

F1 metric of **continuous** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC	
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable
sachs_c	0.504	0.359	0.565	0.463	0.565	0.383
covid_c	0.434	0.464	0.465	0.554	0.378	0.295
building_c	0.829	0.803	0.952	0.925	0.716	0.839
magic-niab_c	0.643	0.700	0.713	0.703	0.616	0.617
magic-irri_c	0.549	0.585	0.592	0.609	0.528	0.544
ecoli70_c	0.553	0.560	0.590	0.639	0.505	0.574
arth150_c	0.422	0.441	0.441	0.539	0.404	0.473

Table B.20

Normalised BIC score of **continuous** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC		Baseline
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable	True graph
sachs_c	-67.9587	-67.9577	-67.9577	-67.9576	-67.9609	-67.9591	-67.9874
covid_c	-90.0674	-90.0530	-90.0661	-90.0506	-90.0705	-90.0701	-90.1249
building_c	-2.3933	-2.3826	-2.3780	-2.3754	-2.4021	-2.3838	-2.3891
magic-niab_c	-48.9989	-48.9891	-48.9891	-48.9860	-49.0019	-48.9916	-49.2406
magic-irri_c	-76.7411	-76.7399	-76.7422	-76.7149	-76.7451	-76.7361	-77.2543
ecoli70_c	-42.2674	-42.2642	-42.2585	-42.2298	-42.2815	-42.2775	-42.3818
arth150_c	-37.8868	-37.8572	-37.8572	-37.7844	-37.8956	-37.8694	-38.6545

Table B.21

Normalised log-likelihood of **continuous** networks learnt using standard and stable HC and Tabu algorithms with different exploration strategies. Best values across Tabu variants, and across HC variants are denoted by bold text, and worst values by bold red text.

Network	Tabu				HC		Baseline
	Standard (unstable)	Tabu-Stable	Tabu-Stable6	Tabu-StableAdapt	Standard (unstable)	HC- Stable	True graph
sachs_c	-67.7511	-67.7509	-67.7509	-67.7510	-67.7583	-67.7643	-67.7821
covid_c	-89.6879	-89.6798	-89.6842	-89.6814	-89.6876	-89.6832	-89.6232
building_c	-1.6840	-1.6834	-1.6840	-1.6807	-1.6831	-1.6847	-1.7450
magic-niab_c	-48.2959	-48.3042	-48.3042	-48.2976	-48.2981	-48.3054	-48.1161
magic-irri_c	-75.9513	-75.9507	-75.9454	-75.9058	-75.9550	-75.9597	-75.8025
ecoli70_c	-41.2728	-41.2737	-41.2723	-41.2786	-41.2814	-41.2846	-41.3535
arth150_c	-36.0027	-35.9402	-35.9402	-35.9045	-36.0130	-35.9537	-36.1481

Data availability

Data will be made available on request.

References

- [1] D. Colombo, M.H. Maathuis, Order-independent constraint-based causal structure learning, *J. Mach. Learn. Res.* 15 (2014) 3921–3962.
- [2] N.K. Kitson, A.C. Constantinou, Eliminating variable order instability in greedy score-based structure learning, in: *International Conference on Probabilistic Graphical Models*, PMLR, 2024, pp. 147–163.
- [3] D. Koller, N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*, MIT Press, 2009.
- [4] E. Kyrimi, S. McLachlan, K. Dube, M.R. Neves, A. Fahmi, N. Fenton, A comprehensive scoping review of Bayesian networks in healthcare: past, present and future, *Artif. Intell. Med.* 117 (2021) 102108.
- [5] B. Cai, L. Huang, M. Xie, Bayesian networks in fault diagnosis, *IEEE Trans. Ind. Inform.* 13 (5) (2017) 2227–2240.
- [6] C. Vitolo, M. Scutari, M. Ghalaieny, A. Tucker, A. Russell, Modeling air pollution, climate, and health data using Bayesian networks: a case study of the English regions, *Earth Space Sci.* 5 (4) (2018) 76–88.
- [7] J. Pearl, *Causality: Models, Reasoning, and Inference*, Cambridge University Press, 2009.
- [8] N.K. Kitson, A.C. Constantinou, The impact of variable ordering on Bayesian network structure learning, *Data Min. Knowl. Discov.* (2024) 1–25.
- [9] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [10] T. Verma, J. Pearl, Equivalence and synthesis of causal models, in: *Proceedings of the Sixth Annual Conference on Uncertainty in Artificial Intelligence*, 1990, pp. 255–270.
- [11] P. Spirtes, C. Glymour, An algorithm for fast recovery of sparse causal graphs, *Soc. Sci. Comput. Rev.* 9 (1) (1991) 62–72.
- [12] D. Margaritis, S. Thrun, Bayesian network induction via local neighborhoods, *Adv. Neural Inf. Process. Syst.* 12 (1999).
- [13] I. Tsamardinos, C.F. Aliferis, A. Statnikov, Time and sample efficient discovery of Markov blankets and direct causal relations, in: *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2003, pp. 673–678.
- [14] P. Spirtes, C. Glymour, R. Scheines, *Causation, Prediction, and Search*, MIT Press, 2001.
- [15] D. Dor, M. Tarsi, A simple algorithm to construct a consistent extension of a partially oriented graph, Technical Report R-185, Cognitive Systems Laboratory, UCLA, 1992, p. 45.
- [16] E. Herskovits, G.F. Cooper, An entropy-driven system for construction of probabilistic expert systems from databases, in: *Proceedings of the Sixth Annual Conference on Uncertainty in Artificial Intelligence*, 1990, pp. 117–128.
- [17] R.R. Bouckaert, Bayesian belief networks: from construction to inference, Ph.D. thesis, University of Utrecht, 1995.
- [18] D.M. Chickering, Optimal structure identification with greedy search, *J. Mach. Learn. Res.* 3 (Nov) (2002) 507–554.
- [19] J. Ramsey, M. Glymour, R. Sanchez-Romero, C. Glymour, A million variables and more: the fast greedy equivalence search algorithm for learning high-dimensional graphical causal models, with an application to functional magnetic resonance images, *Int. J. Data Sci. Anal.* 3 (2017) 121–129.
- [20] M. Bartlett, J. Cussens, Integer linear programming for the Bayesian network structure learning problem, *Artif. Intell.* 244 (2017) 258–271.
- [21] I. Tsamardinos, L.E. Brown, C.F. Aliferis, The max-min hill-climbing Bayesian network structure learning algorithm, *Mach. Learn.* 65 (1) (2006) 31–78.
- [22] M. Gasse, A. Aussem, H. Elghazel, A hybrid algorithm for Bayesian network structure learning with application to multi-label learning, *Expert Syst. Appl.* 41 (15) (2014) 6755–6772.
- [23] J. Peters, J.M. Mooij, D. Janzing, B. Schölkopf, Causal discovery with continuous additive noise models, *J. Mach. Learn. Res.* 15 (1) (2014) 2009–2053.
- [24] X. Zheng, B. Aragam, P.K. Ravikumar, E.P. Xing, Dags with no tears: continuous optimization for structure learning, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [25] N.K. Kitson, A.C. Constantinou, Z. Guo, Y. Liu, K. Chobtham, A survey of Bayesian network structure learning, *Artif. Intell. Rev.* (2023) 1–94.
- [26] J. Suzuki, Learning Bayesian belief networks based on the minimum description length principle: basic properties, *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* 82 (10) (1999) 2237–2245.
- [27] D. Heckerman, D. Geiger, D.M. Chickering, Learning Bayesian networks: the combination of knowledge and statistical data, *Mach. Learn.* 20 (3) (1995) 197–243.
- [28] C. Yuan, B. Malone, X. Wu, Learning optimal Bayesian networks using A* search, in: *Twenty-Second International Joint Conference on Artificial Intelligence*, 2011, pp. 2186–2191.
- [29] G.F. Cooper, E. Herskovits, A Bayesian method for the induction of probabilistic networks from data, *Mach. Learn.* 9 (4) (1992) 309–347.
- [30] P. Larranaga, C.M. Kuijpers, R.H. Murga, Y. Yurramendi, Learning Bayesian network structures by searching for the best ordering with genetic algorithms, *IEEE Trans. Syst. Man Cybern., Part A, Syst. Hum.* 26 (4) (1996) 487–493.
- [31] S. Behjati, H. Beigy, Improved K2 algorithm for Bayesian network structure learning, *Eng. Appl. Artif. Intell.* 91 (2020) 103617.
- [32] N. Friedman, D. Koller, Being Bayesian about network structure. A Bayesian approach to structure discovery in Bayesian networks, *Mach. Learn.* 50 (2003) 95–125.
- [33] B.M. Broom, K.-A. Do, D. Subramanian, Model averaging strategies for structure learning in Bayesian networks with limited data, *BMC Bioinform.* 13 (2012) 1–18.
- [34] A. Constantinou, N.K. Kitson, Y. Liu, K. Chobtham, A.H. Amirkhizi, P.A. Nanavati, R. Mbuvha, B. Petrungaro, Open problems in causal structure learning: a case study of COVID-19 in the UK, *Expert Syst. Appl.* 234 (2023) 121069.
- [35] M. Scutari, C.E. Graafland, J.M. Gutiérrez, Who learns better Bayesian network structures: accuracy and speed of structure learning algorithms, *Int. J. Approx. Reason.* 115 (2019) 235–253.
- [36] M. Leonelli, bnRep: a repository of Bayesian networks from the academic literature, arXiv preprint, arXiv:2409.19158, 2024.
- [37] A.C. Constantinou, Y. Liu, K. Chobtham, Z. Guo, N.K. Kitson, The Bayesys data and Bayesian network repository v1.5, <http://bayesian-ai.eecs.qmul.ac.uk/bayesys/>, Bayesian Artificial Intelligence Research Lab., Queen Mary University of London, London, UK, 2024.
- [38] M. Scutari, Bayesian network repository, <https://www.bnlearn.com/bnrepository/>, 2021.
- [39] A.C. Constantinou, Evaluating structure learning algorithms with a balanced scoring function, arXiv preprint, arXiv:1905.12666, 2019.
- [40] M. Scutari, bnlearn (Version 4.7) [Computer program], <https://cran.r-project.org/web/packages/bnlearn/index.html>, 2021 (downloaded: 17 December 2021).
- [41] J.D. Ramsey, K. Zhang, M. Glymour, R.S. Romero, B. Huang, I. Ebert-Uphoff, S. Samarasinghe, E.A. Barnes, C. Glymour, Tetrad—a toolbox for causal discovery, in: *8th International Workshop on Climate Informatics*, 2018, pp. 1–4.